4. Adaptive Query Execution

Spark 1.X :- Introduced Catalyst Optimizes (Rule Based)
Spark 2.X :- Introduced Cosed based Optimizes
Spark 3.0 :- Introduced Adaptive Query Execution (AQE)

↳ Spark.conf.set("spark.sql.adaptive.enabled", True)
Default Enabled.

AQE in Spark is a runtime optimization technique that dynamically modifies the execution plan based on actual ~~Statistics~~ Statistics collected during query execution.

Spark originally plans everything before execution, By catalyst optimizer, based on Estimated data size, Estimated cardinality. but in runtime data get skewed, sizes are wrong, joins behave differently. Which result in performance ~~does~~ getting down.

- AQE is triggered between stages during runtime.

AQE Features
1. Dynamic Join Strategy
2. Skew Join Handling
3. Coalescing Shuffle Partitions

- AQE

5. Shuffling Parameter

Shuffle is exchange data accross executors.

There are two types of transformations.
1. Narrow Transformation - Not Shuffle
2. Wide Transformation.

In wide transformation, a executor need data from another executor(s)

A shuffle requires
1. Disk I/O
2. Network I/O
3. Data Serialization / Deserialization

What is shuffling parameter :- While exchanging the data in between stages, how many number of partitions to be created, that is called shuffling parameter.

- Default Number : - 200

```
spark.conf.get("spark.sql.shuffle.partitions")
```

- Factors to consider while Choosing shuffling params

1. For smaller dataset, 200 partitions would split the data into much smaller chunks which add memory and network overhead
2. For large dataset, 200 partions would great bigger chunck of data which lead to poor parallelism.

Specious Trade)

- Make sure that partion size would be at least 128 MB to 200 MB
- Make sure the number of partitions are multiple of number of cores

6. Partion By

In Spark, partitionBy is about how to write data in disk storage, It not about how it's processed in memory.

- partitionBy exist to avoid data scanning of irrelevant data by separating files so Spark can skip work of them during queries.

1. df.write.partitionBy(key).csv(path)
2. .partitionBy("key1", "key2")
3. .option("maxRecordsPerFile", 4200).partitionBy()

How files are stored in each case

7. Dataframe Checkpoint

- Checkpoint is process of materializing the dataframe resultant data into storage directory.
- It is different from Cache.
- It says "Forget how this was computed, just store the result safely in disk".

- spark.sparkContext.setCheckpointDir("/path")

```
df = df.filter(...).groupBy(..)
df = df.checkpoint()
df.count()
```

Checkpoint	Cache
• Write in disc • Truncates DAG	• Write into memory • Retain DAG

To ~~def~~ we can define where to store checkpoint.

8. Data Skew

- Data is unevenly distributed among partitions in cluster.

- It downgrade the performance, A single executor may work for an houre and other just finish the task in 1 second.

*How to handle Data Skew

1. Salt the Skewed column :-
2. Apply Skew-hint :- They give hints to Spark to create better query plan. one that does not suffer from data skew

3. AQE can balance Out the partition for automatically :- Dynamically Optimize Skew join.

* SELECT SKEW ('order'